

StockMind AI — Complete Application Documentation

Version 0.2.0 | Generated May 08, 2026 | Local-first AI-powered stock market prediction platform

1. Application Overview

StockMind AI is a fully local, device-resident AI-powered stock market prediction and analysis platform. It runs entirely on a single laptop or mobile device with no cloud dependency, no external hosting, and no internet account required for core functionality.

PURPOSE: Generate calibrated probability predictions for Indian and global financial instruments. NOT financial advice. NOT trade execution. Predictions are probabilistic signals for informational use only.

KEY CHARACTERISTICS:

- Local-first: all data stored encrypted on device (AES-256-GCM)
- Single command startup: `node start.js --dev`
- Three-factor authentication: username + password + 12-digit HMAC key
- Dual AI engine: Python ML backend (LightGBM/XGBoost/LSTM) with JS fallback
- Real-time market data: Yahoo Finance v8 API (free, no key) + Binance WebSocket
- 7 market modules: Indian Indices, Equities, F&O, Crypto, Forex, Commodities, Global Indices
- 16 calibrated signals per prediction request, sorted highest to lowest probability
- Adaptive learning: tracks prediction outcomes vs live prices, adjusts weights
- Walk-forward backtesting: 3-year historical validation (75-97% accuracy gate)

2. Technology Stack

2.1 Frontend

Framework: React 19.2.0 + Vite 7.2.4

Routing: react-router-dom 7.6.0

State: Zustand 5.0.5

Charts: Recharts 2.15.3

Styling: CSS Modules (component-scoped) + global design tokens in index.css

Build output: build/ directory (ESBuild minification)

Dev server: `http://localhost:3000` (Vite HMR)

Prod server: `http://localhost:5000` (served by Express)

Path aliases: `@components`, `@pages`, `@hooks`, `@services`, `@store`, `@utils`, `@assets`, `@types`

2.2 Backend (Node.js / Express)

Runtime: Node.js v18+ (ESM modules)

Framework: Express 4.21.2

Auth: Argon2id password hashing (argon2 0.43.0), HMAC-SHA256 key verification

Security: Helmet 8.0.0 (CSP), CORS (localhost only), express-rate-limit 7.5.0

Port: 5000 (configurable via PORT env var)

Sessions: In-memory Map (no JWT, no cookies) — cleared on server restart

Data: AES-256-GCM encrypted JSON files in data/ directory

Market data: yahoo-finance2 (server-side, no CORS issues)

2.3 Python AI Backend

Framework: FastAPI 0.115.5 + Uvicorn 0.32.1

Port: 8001

ML Models: LightGBM 4.5.0, XGBoost 2.1.3, LSTM (PyTorch stub), FinBERT (stub)

Features: ~80 engineered features across 6 buckets (Price Action, Volatility, Trend/Momentum, Volume, Market Context, Time-Based)

Calibration: 3-stage pipeline: Platt scaling → Isotonic regression → Regime adjustment

Backtest: Walk-forward (train 500 bars, test 60 bars, step 30 bars)

Data fetch: httpx 0.27.2 for direct Yahoo Finance v8 API calls

Fallback: If Python backend unavailable, JS prediction engine activates automatically

3. Project Structure

ROOT/

start.js Unified launcher (all 3 processes, parallel startup)

package.json Node.js dependencies and npm scripts

vite.config.js Vite build config, path aliases, proxy

.env.example All environment variables documented

users-seed.json First-run user creation (edit before first start)

GETTING-STARTED.md Setup and login guide

src/ React frontend source

App.jsx Router + lazy-loaded page imports

index.css Global design tokens (dark/light theme CSS vars)

main.jsx React entry point

components/ Shared reusable UI components

pages/ Route-level page components (one folder per route)

hooks/ Custom React hooks

services/ API client wrappers (no React, no side effects)

store/ Zustand global state slices

utils/ Pure functions, constants, helpers

types/ JSDoc type definitions

server/ Node.js Express backend

index.js Entry point, middleware, route registration

routes/ Express route handlers (thin, delegate to services)

services/ Business logic (auth, predictions, market data)

middleware/ Auth verification middleware

storage/ AES-256-GCM file I/O

keygen/ Standalone key generator scripts

utils/ Server-side sanitization

ai_backend/ Python FastAPI ML inference server

main.py FastAPI app, all endpoints

engine/

dispatcher.py Routes requests to correct signal generator

features.py Feature engineering pipeline (~80 features)

models.py LightGBM, XGBoost, LSTM, FinBERT wrappers

registry.py Ensemble meta-learner, SHAP reason generation

calibration.py 3-stage probability calibration

backtest.py Walk-forward backtest engine

data_fetcher.py OHLCV data from request or Yahoo Finance

health.py System health endpoint data

requirements.txt Python dependencies (pinned versions)

data/ Encrypted local storage (AES-256-GCM)
users/ Per-user encrypted JSON records + index
predictions/ Monthly partitioned predictions (YYYY-MM/)
backtest/ Backtest result cache
system/ .salt file for key derivation

4. Authentication System

Three-factor authentication flow:

STEP 1 — Username + Password

- Argon2id verification (64 MiB memory, 3 iterations, 4 parallelism)
- Brute-force lockout: 5 failed attempts → 15-minute lockout
- Returns: stepToken (64-char hex, valid 10 minutes)
- mustChangePassword flag returned if admin set temp password

STEP 2 — 12-Digit Access Key (XXXX-XXXX-XXXX)

- HMAC-SHA256 verification: hmac(key_digits, userId) compared with stored hash
- timingSafeEqual used for all comparisons
- Key validity: 7 days max (configurable, set at generation time)
- Returns: sessionToken (64-char hex) + user object

SESSION

- Stored in server memory (Map) — cleared on server restart
- TTL: 8 hours, sliding expiry (activity extends by 30 min)
- Stored in browser sessionStorage — cleared on tab close
- No JWT, no cookies

PASSWORD REQUIREMENTS

- Minimum 8 characters
- At least 1 uppercase letter (A-Z)
- At least 1 number (0-9)
- At least 1 special character (!@#\$%^&* etc.)

IN-UI KEY GENERATION (Login Step 2 page)

- "Get Key" button opens inline panel
- If stepToken available: Quick Generate (no re-auth needed)
- If no stepToken: asks username + password inline
- Key shown masked, hold-to-reveal, copy button
- Auto-fills key input field, auto-clears after 60 seconds
- Previous key invalidated on new key generation

ADMIN CAPABILITIES

- Create users (mustChangePassword = true by default)
- Generate keys for any user (shown once, share securely)
- Reset passwords (user must change on next login)
- Deactivate/reactivate users (immediate session invalidation)
- View all users: status, last login, login count, key expiry

5. Backend API Endpoints

5.1 Auth Routes /api/auth

POST /login/step1 Username + password -> stepToken

POST /login/step2 stepToken + 12-digit key -> sessionToken + user

POST /logout Invalidate session

GET /me Current session info

POST /keygen Generate key (requires full session + password re-auth)

POST /keygen-with-step-token Generate key using stepToken (login page, Quick Generate)

POST /keygen-with-credentials Generate key using username+password (login page, no stepToken)

POST /change-password Self-service password change (requires current password)

Rate limits:

/login/* 10 requests per 15 minutes

/keygen* 5 requests per hour

/api/* 300 requests per minute

5.2 User Routes /api/users (admin only except preferences)

GET / List all users

POST / Create user

PATCH /:userId/preferences Update preferences (self or admin)

PATCH /:userId/activate Reactivate deactivated user

DELETE /:userId Deactivate user

POST /:userId/keygen Generate key for user

POST /:userId/reset-password Admin reset password

5.3 Market Routes /api/market

GET /quote/:symbol?exchange=NSE Single quote (15s cache)

GET /batch?symbols=A,B&exchange=NSE Batch quotes (max 30, 300ms delay between)

GET /ohlcv/:symbol?exchange=NSE&bars=250 Daily OHLCV history (5min cache, up to 1000 bars)

GET /search?q=query Symbol search

GET /status Yahoo Finance reachability check

Data sources (priority order):

1. Yahoo Finance v8 chart API (direct HTTP, less rate-limited)

2. yahoo-finance2 library (fallback)

3. Stale cache (last known value, marked stale:true)

5.4 Prediction Routes /api/predictions

POST / Save prediction batch

GET /:symbol Recent predictions (limit param)

GET /:symbol/accuracy Rolling accuracy stats (days param)

GET /:symbol/backtest Cached backtest results

POST /:symbol/backtest Save backtest result

GET /:symbol/versions Model version history (last 3)

GET /:symbol/deviations Learning deviation history

GET /:symbol/adaptive-weight Adaptive weight for instrType

PATCH /:symbol/:id/outcome Resolve prediction outcome

POST /cleanup Run cleanup (admin only)

5.5 Inference Routes /api/inference

POST /predict Generate 16 signals (proxies to Python AI, falls back to JS)
GET /health Python AI backend health
GET /models/status Model load status
POST /backtest Trigger walk-forward backtest
POST /calibrate Manual recalibration trigger

The inference route enriches requests before sending to Python:

- Fetches 3 years of real OHLCV from Yahoo Finance
- Computes adaptive weight from deviation history
- Passes predictionMode, futuresMeta, optionMeta

5.6 Strategy Routes /api/strategies

POST /parse NLP strategy parser (plain English -> prediction params)
GET / List user's saved strategies
POST / Save a strategy
DELETE /:id Delete a strategy
GET /:id/backtest Run backtest for a saved strategy

5.7 Other Routes

GET /api/health System health (ECE, Brier score, feed status)
GET /api/validator/status Outcome validator status
GET /api/validator/events SSE stream: outcome_resolved, accuracy_drift, accuracy_update
GET /api/zerodha/login Zerodha Kite Connect OAuth flow
GET /api/zerodha/callback OAuth callback

6. Python AI Backend Endpoints (port 8001)

GET /health System health, model load status
POST /predict Generate 16 signals (spot/futures/options/deriv-rec)
POST /backtest Walk-forward backtest (returns accuracy, Sharpe, drawdown)
POST /calibrate Recalibrate on accuracy drift (3 action tiers)
GET /models/status Per-model load status (real vs mock)
GET /adaptive-weight/{symbol} Adaptive weight info (computed by Node.js)
POST /strategy/parse AI-enhanced strategy parser

PREDICTION REQUEST BODY:

symbol, exchange, instrType (spot/futures/options)
basePrice, capital, riskPct, direction (long/short/both)
predictionMode (learning/realworld/both)
adaptiveWeight (0.5-2.0, from Node.js deviation history)
ohlcV (list of {date,open,high,low,close,volume} — pre-fetched by Node.js)
optionMeta (strike, optType, daysLeft, iv, lotSize)
futuresMeta (futuresPrice, basis, lotSize, daysLeft, series)
isDerivRec / isIndexDerivRec (derivative recommender mode)

PREDICTION RESPONSE:

requestId, symbol, exchange, generatedAt, modelVersion
predictionMode, adaptiveWeight, dataSource (real/mock)

signals[] (16 items, sorted highest->lowest probability)
suppressedCount

7. Signal Structure (per signal)

Each of the 16 signals contains:

IDENTIFICATION:

rank (1-16), id (UUID hex), type (LONG/SHORT)

instrType (spot/futures/options), symbol

PRICE LEVELS:

entryPrice, entryZoneLow, entryZoneHigh

t1Price, t2Price, t3Price

stopLoss, immediateOptimalSL

maxRisk (in currency)

PROBABILITIES (all clamped 5%-99%):

probability (overall signal probability)

t1Probability, t2Probability, t3Probability

slProbability

RISK METRICS:

riskRewardRatio, validityBars, validity (ISO timestamp)

GRADING:

grade: A+ (>=80%), A (>=70%), B (>=60%), C (>=50%), D (<50%)

OPTIONS-SPECIFIC:

optType (CE/PE), strike, iv (implied volatility %)

delta, gamma, theta, vega (Greeks)

breakeven, indexMoveNeeded, probReachBreakeven

lotSize, lotCount

FUTURES-SPECIFIC:

basis, lotSize, lotCount, daysLeft, series (Near/Mid/Far)

METADATA:

regime, reasons[] (up to 6 SHAP-derived explanations)

suppressed (bool), suppressReason

isNextDay (bool), nextTradingDay

dataSource (real/mock), predictionMode

8. Market Modules

7 modules, each self-contained with its own data service and prediction config:

MODULE ID LABEL EXCHANGE INSTRUMENTS

indices-india Indian Indices NSE NIFTY50, BANKNIFTY, FINNIFTY, SENSEX, MIDCPNIFTY, NIFTYNXT50

equities-india Indian Equities NSE/BSE All NSE/BSE listed stocks

fno-india F&O NSE Futures + Options (CE/PE) for indices and stocks

crypto Crypto BINANCE BTC, ETH, BNB, SOL, XRP, ADA, DOGE, AVAX, DOT, MATIC

forex Forex FOREX USDINR, EURUSD, GBPUSD, USDJPY, AUDUSD, USDCHF, EURINR
commodities Commodities MCX/COMEX GOLD, SILVER, CRUDEOIL, NATURALGAS, COPPER, ALUMINIUM
global-indices Global Indices GLOBAL SPX, NDX, DJI, FTSE, DAX, N225, HSI

DERIVATIVE RECOMMENDER (indices-india, global-indices):

Scans ATM $\pm$ 4 strikes (CE + PE) + futures

Ranks by: probability $\times$ R:R $\times$ capital efficiency

Returns top 16 across all instruments

9. Feature Engineering (AI Backend)

~80 features across 6 buckets:

BUCKET 1 — Price Action:

log_return_1, log_return_5, log_return_20

candle_body, upper_wick, lower_wick, gap, direction_streak

BUCKET 2 — Volatility:

atr14_pct, atr5_pct, parkinson_vol

bb_width (Bollinger Band width), bb_position

BUCKET 3 — Trend & Momentum:

ema5/10/20/50/200 slope and price_vs_ema

rsi14 (normalised 0-1)

macd_hist, macd_hist_vel

adx14 (normalised 0-1)

BUCKET 4 — Volume:

volume_ratio (vs 20-day SMA), volume_climax

obv_slope5 (On-Balance Volume 5-bar slope)

vwap_dev (VWAP deviation)

BUCKET 5 — Market Context:

trend_strength (price vs EMA50)

ema_cross_20_50, price_compression

stoch_k (Stochastic %K), williams_r

cci20 (CCI normalised)

pct_from_52w_high, pct_from_52w_low

BUCKET 6 — Time-Based:

hour_sin, hour_cos, dow_sin, dow_cos, month_sin, month_cos

10. UI Pages and Components

10.1 Pages (Routes)

ROUTE PAGE DESCRIPTION

/login LoginPage 3-factor login (Step 1: credentials, Step 2: key)

Inline keygen panel on Step 2 (no terminal needed)

/dashboard DashboardPage Market overview, 7 module cards, live indices

/predictions PredictionsPage 16-signal generator with dual-mode (Learning/Real-World)
PredictionModeSelector, RefreshCountdown (60s auto-refresh)
PredictionInputPanel, SignalGrid, PredictionChart
LivePricePanel, OptionsPanel, FuturesPanel
/backtest BacktestPage Walk-forward backtest runner
Strategy Builder (plain English -> prediction params)
My Strategies (saved strategies with backtest results)
/strategies StrategiesPage Library of 8 open strategies to learn from
Filter by category, instrument, complexity, accuracy
Apply to Predictions button
/favourites FavouritesPage Saved symbols in tabs (stocks, options, futures, etc.)
/history HistoryPage Prediction history + 30-day accuracy stats per symbol
/settings SettingsPage Trading defaults, data retention, tooltip preferences
/admin AdminPage User management (admin only)
Add user, generate keys, reset passwords, deactivate

10.2 Key Components

AppShell Main layout: topbar, sidenav, mobile bottom nav
Topbar: logo, indices ticker, market selector, theme toggle, night light (0-100%), keygen button, change-password button
Sidenav: 7 nav items + admin link
Mobile: fixed bottom nav bar (44px touch targets, WCAG AA)
SignalGrid 16 prediction cards, mobile-first
Probability bar, R:R visual bar, price levels table
Greeks (options), breakeven context, expand reasons
OptionsPanel Full options chain with BS-priced premiums
Moneyness filter (ITM/ATM/OTM/ALL), expiry selector
CE/PE toggle, Greeks display, breakeven context
Click any chain row to select strike instantly
FuturesPanel Futures contract picker (Near/Mid/Far month)
Basis display (cost-of-carry model), lot count selector
Contract value + ~12% margin estimate
Near-expiry warning (<= 5 days)
PredictionModeSelector Learning / Real-World / Dual Mode toggle
Accuracy stats from deviation history
Adaptive weight indicators per instrument type
RefreshCountdown 60-second auto-refresh bar with force-refresh and pause
Only runs during market hours (crypto: always)
InfoTooltip Reusable info icon with overlay panel
Keyboard accessible, click-outside dismiss
Per-page on/off control, show/hide examples toggle
Available on: Backtest, Strategies, Predictions pages
KeygenModal In-app key generation (topbar button)
Password re-verification, validity selector
Hold-to-reveal, copy, 60s auto-clear

ChangePasswordModal Self-service password change
Real-time strength bar (Weak/Fair/Good/Strong)
Live requirements checklist
Forced mode for first-login (mustChangePassword)

IndicesTicker Scrolling ticker with live index values
Hover tooltip shows OHLC, volume, change %

LivePricePanel Real-time price for selected instrument
LIVE badge (market open) / EOD badge (market closed)

Disclaimer Non-removable, non-dismissible
Jurisdiction-aware (IN/US/EU via VITE_DISCLAIMER_JURISDICTION)

10.3 State Management (Zustand Stores)

authStore token, user, isAuthenticated
setSession, clearSession, updatePreferences, clearMustChangePassword

marketStore activeModuleId, activeSymbol, marketModules list
setActiveModule, setActiveSymbol, getActiveModule, getActiveSymbolMeta

themeStore theme (dark/light), nightLight (0-100%)
toggleTheme, setNightLight
Persisted to localStorage

predictionModeStore activeMode (learning/realworld/both), autoRefresh
deviationHistory (last 500 outcomes)
adaptiveWeights per instrType (spot/futures/options)
learningStats (totalPredictions, correctPredictions, avgDeviation)
Persisted to localStorage

uiPrefsStore infoTooltips (global on/off), showExamples
pageTooltips (per-page override map)
Persisted to localStorage

tooltipStore (legacy tooltip state)

11. Prediction Modes

Two modes run simultaneously:

LEARNING MODE

Purpose: Exploratory — tracks deviation from actual, adapts weights
Behaviour: Adds controlled noise to probability scores (wider confidence intervals)
Use case: Building up deviation history, discovering what works
Icon: Brain emoji

REAL-WORLD MODE

Purpose: Calibrated — uses learned weights for maximum accuracy
Behaviour: Applies adaptive weight multiplier (0.8-1.2) to probability scores
Use case: Live trading decisions
Icon: Target emoji

DUAL MODE (default)

Blends both: 40% learning + 60% real-world

Both run simultaneously to learn and adapt

ADAPTIVE WEIGHT CALCULATION:

- Tracks last 500 prediction outcomes per symbol per instrType
- Computes accuracy over last 20 outcomes
- $\text{Weight} = 0.8 + (\text{accuracy} * 0.4)$
- accuracy < 50% -> weight 0.8 (reduce confidence)
- accuracy > 80% -> weight 1.2 (boost confidence)
- Stored in predictionModeStore (localStorage) and server deviation store

AUTO-REFRESH:

- Every 60 seconds during market hours
- Crypto: always refreshes (24/7 market)
- Countdown bar with force-refresh and pause buttons
- Re-uses last prediction parameters automatically

12. Outcome Validator

Background service running on the Node.js server.

Polls every 2.4 seconds during market hours (NSE: Mon-Fri 09:15-15:30 IST).

WHAT IT DOES:

1. Loads all unresolved predictions from last 30 days
2. Fetches current live price for each symbol (Yahoo Finance -> Finnhub fallback)
3. Checks if T1/T2/T3 or SL has been hit (or validity expired -> TIMEOUT)
4. Resolves outcome, writes back to encrypted storage
5. Saves deviation record for adaptive learning
6. Recomputes rolling accuracy (30-day window)
7. Notifies Python AI backend if accuracy drifts > 3%
8. Broadcasts real-time events via SSE to connected frontend clients

OUTCOME TYPES:

T1_HIT, T2_HIT, T3_HIT -> correct (wasCorrect: true)

SL_HIT -> incorrect

TIMEOUT -> expired without hitting any level

P&L CALCULATION:

Correct: $\text{abs}(\text{outcomePrice} - \text{entryPrice}) * \text{lotCount} * \text{lotSize}$

Incorrect: $-\text{abs}(\text{entryPrice} - \text{stopLoss}) * \text{lotCount} * \text{lotSize}$

Commission: $\text{lotCount} * 40$ (approx Rs 40 per lot round trip)

SSE EVENTS (GET /api/validator/events):

outcome_resolved -> { symbol, id, outcome, price, wasCorrect, pnlAfterCommission }

accuracy_drift -> { symbol, prev, current, drift }

accuracy_update -> { symbol, accuracyPct, totalResolved, stable }

ACCURACY GATE:

Stable range: 75% - 97%

Below 75%: action = retrain_required

Above 97%: action = check_overfitting

Python AI backend notified for recalibration on drift > 3%

13. Backtest Engine

Walk-forward methodology (Python AI backend):

PARAMETERS:

Training window: 500 bars (~2 years daily)

Test window: 60 bars (~3 months)

Step size: 30 bars (1 month roll-forward)

Signal frequency: Every 5 bars in test window

SIGNAL SIMULATION:

ATR-based levels (SL = 1.3x ATR, T1 = 1.8x ATR)

Simulates against next 5 bars (high/low check)

Outcomes: T1_HIT, T2_HIT, T3_HIT, SL_HIT, TIMEOUT

METRICS RETURNED:

accuracyPct, stable (bool)

t1HitRate, t2HitRate, t3HitRate, slHitRate

avgRR (average risk-reward ratio)

avgPnlPct (average P&L as % of entry)

sharpe (simplified Sharpe ratio)

maxDrawdownPct (peak-to-trough loss)

signalsTested (total signals simulated)

barsUsed, dataSource (real/mock)

STABILITY GATE:

75% <= accuracy <= 97% -> stable: true

Below 75% -> action: retrain_required

Above 97% -> action: check_overfitting

DATA SOURCE:

Node.js pre-fetches 3 years of OHLCV from Yahoo Finance

Passes as ohlcv[] array in request body

Python uses real data if provided, mock if not

14. Data Storage

All data stored in data/ directory, AES-256-GCM encrypted.

ENCRYPTION:

Algorithm: AES-256-GCM (authenticated encryption)

Key: PBKDF2(DATA_PASSWORD, salt, 100000 iterations, SHA-256, 32 bytes)

Salt: Random 32 bytes, stored in data/system/.salt (generated once)

IV: Random 12 bytes per file write

Layout: [IV(12)] [AuthTag(16)] [Ciphertext]

Extension: .enc

DATA LAYOUT:

data/users/index.enc username -> userId mapping

data/users/{userId}.enc user record (passwordHash, keyHash, preferences)

data/predictions/YYYY-MM/ monthly partitioned predictions

{SYMBOL}_{ts}_{id}.enc full signal payload

summary.csv append-only CSV (not encrypted)
data/predictions/versions/ model version history per symbol
data/predictions/deviations/ adaptive learning deviation records
data/backtest/ backtest result cache
data/system/.salt PBKDF2 salt (never delete)

CLEANUP SCHEDULER:

Runs every 24 hours

Deletes .enc files older than CLEANUP_RETENTION_DAYS (default: 30)

Keeps summary.csv files intact

Configurable via CLEANUP_RETENTION_DAYS env var

COMPRESSION:

Predictions stored as compact JSON (no pretty-print in production)

CSV summaries for space-efficient historical reference

15. Market Data Sources

SOURCE TYPE KEY NEEDED COVERAGE

Yahoo Finance v8 REST/HTTP No All instruments (primary)

yahoo-finance2 Library No All instruments (fallback)

Binance WebSocket WebSocket No Crypto (real-time)

Zerodha Kite WebSocket Yes (paid) Indian markets (optional)

Finnhub REST Optional Price fallback for validator

YAHOO FINANCE SYMBOL MAPPING:

NIFTY50 -> ^NSEI BANKNIFTY -> ^NSEBANK

SENSEX -> ^BSESN NIFTYIT -> ^CNXIT

FINNIFTY -> NIFTY_FIN_SERVICE.NS

BTCUSDT -> BTC-USD ETHUSDT -> ETH-USD

GOLD -> GC=F CRUDEOIL -> CL=F

SPX -> ^GSPC NDX -> ^NDX

NSE stocks -> {SYMBOL}.NS BSE stocks -> {SYMBOL}.BO

CACHE TTL:

Quotes: 15 seconds

OHLCV: 5 minutes

Max cache size: 500 entries (LRU eviction)

LIVE vs EOD:

marketState == REGULAR -> LIVE badge, real-time price

marketState == CLOSED -> EOD badge, last close price

Predictions when closed: based on EOD close, flagged isNextDay: true

16. Environment Variables

FRONTEND (VITE_ prefix, available in browser):

VITE_LIVE_FEED auto | zerodha | mock (default: auto)

VITE_ZERODHA_API_KEY Kite Connect public API key

VITE_FINNHUB_KEY Finnhub API key (optional)

VITE_BINANCE_WS_ENABLED true/false (default: true)
VITE_AI_API_URL http://localhost:5000/api/inference
VITE_APP_ENV development | production
VITE_APP_VERSION 0.2.0
VITE_ENABLE_PREDICTIONS true/false feature flag
VITE_ENABLE_SENTIMENT true/false (FinBERT, default: false)
VITE_ENABLE_CV_PATTERNS true/false (default: false)
VITE_ENABLE_RL_TIMING true/false (default: false)
VITE_DISCLAIMER_JURISDICTION IN | US | EU (default: IN)
VITE_CONFIDENCE_FLOOR 5 (minimum probability %)
VITE_CONFIDENCE_CEILING 99 (maximum probability %)
VITE_MIN_HISTORY_MONTHS 6

BACKEND (server-side only):
DATA_PASSWORD AES-256-GCM encryption password (MUST be consistent)
Default: stockmind-local-dev-password
PORT Express port (default: 5000)
AI_BACKEND_URL http://localhost:8001
CLEANUP_RETENTION_DAYS 30 (days to keep predictions)
FINNHUB_KEY Server-side Finnhub key for outcome validator
ZERODHA_API_SECRET Kite Connect secret (server-side only)
ZERODHA_ACCESS_TOKEN Auto-filled after OAuth flow
ZERODHA_REDIRECT_URL http://localhost:5000/api/zerodha/callback

17. npm Scripts

STARTUP:

npm start node start.js (production)
npm run start:dev node start.js --dev (Vite HMR)
npm run start:build node start.js --build (build then start)
npm run start:no-ai node start.js --no-ai (skip Python)

DEVELOPMENT:

npm run dev concurrently: vite + node server/index.js
npm run dev:frontend vite only
npm run dev:backend node server/index.js only
npm run build vite build (output: build/)
npm run preview vite preview

USER MANAGEMENT:

npm run keygen <username> Generate 12-digit access key
npm run adduser Interactive user creation (terminal)
npm run reseed Re-run users-seed.json

CODE QUALITY:

npm run lint eslint --max-warnings=0
npm run format prettier --write .
npm run test vitest --run
npm run test:watch vitest (watch mode)
npm run test:coverage vitest --coverage

18. Security Architecture

AUTHENTICATION:

- Argon2id (memory-hard): 64 MiB, 3 iterations, 4 parallelism
- Brute-force lockout: 5 attempts -> 15-minute lockout
- timingSafeEqual for all secret comparisons
- Session sliding expiry (8h TTL, 30min slide on activity)
- mustChangePassword flag for admin-created accounts

TRANSPORT:

- CORS: localhost only (regex: /^http:\/\/(localhost|127.0.0.1)(:\d+)?\$/)
- Helmet CSP: allowlist self, Yahoo Finance, Binance WS, Finnhub
- No HTTPS (local-only app)
- Body limit: 512kb

RATE LIMITING:

- Login: 10 requests per 15 minutes
- Keygen: 5 requests per hour
- General API: 300 requests per minute

DATA:

- AES-256-GCM: authenticated encryption (tamper-evident)
- PBKDF2 key derivation: 100,000 iterations
- Passwords: Argon2id hash, never stored plaintext
- Keys: HMAC-SHA256 hash, plaintext shown once and discarded
- Sessions: in-memory only, cleared on server restart

INPUT VALIDATION:

- All user strings through sanitizeTicker() or stripHtml()
- No eval(), new Function(), or innerHTML with user data
- Username: 4-32 chars, [a-z0-9_.-] only
- Password: min 8 chars, uppercase + number + special required
- Token format validated (64 hex chars) before any lookup

DESIGN PRINCIPLES:

- No JWT (sessions in memory)
- No cookies
- No external auth service
- No cloud storage
- Local device only — never expose port 5000 to internet

19. First-Time Setup Guide

PREREQUISITES:

- Node.js v18 or higher (nodejs.org)
- npm v9 or higher (comes with Node.js)
- Python 3.9+ (optional, for AI backend)

STEP 1 — Edit users-seed.json

Open users-seed.json and set your username and password.

Password must have: 8+ chars, uppercase, number, special char.

Example:

```
{ "username": "yourname", "password": "YourPass@123", "role": "admin" }
```

STEP 2 — Install and start

```
npm install --legacy-peer-deps
```

```
node start.js --dev
```

STEP 3 — Generate your access key

In a second terminal:

```
npm run keygen yourname
```

Copy the 12-digit key (shown once only).

STEP 4 — Log in

Open <http://localhost:3000>

Step 1: Enter username + password

Step 2: Enter the 12-digit key (XXXX-XXXX-XXXX)

OR: On Step 2 page, click "Get Key" to generate inline (no terminal needed)

STEP 5 — Change default password

After first login, you will be prompted to change your password

(mustChangePassword = true for admin-created accounts).

DATA PASSWORD:

Default: stockmind-local-dev-password

To change: set DATA_PASSWORD env var BEFORE first run

WARNING: Changing after first run makes all existing data unreadable

20. Strategy Library (Built-in)

8 open strategies available on the Strategies page:

NAME CATEGORY INSTRUMENT ACCURACY COMPLEXITY

RSI Reversal - NIFTY50 Mean Reversion Spot 78% Beginner

Volume Breakout - BANKNIFTY Breakout Futures 76% Intermediate

ATM Call Buy - Low IV Options Options 74% Intermediate

EMA Crossover - Equities Trend Following Spot 75% Beginner

OTM Put Sell - High IV Options Income Options 82% Advanced

MACD Momentum - Crypto Momentum Spot 71% Beginner

VWAP Reversion - NIFTY Mean Reversion Spot 77% Intermediate

Bollinger Squeeze Breakout Breakout Spot 73% Intermediate

Each strategy shows:

- Description and logic explanation
- Technical indicator filters/conditions
- Best market conditions and when to avoid
- Backtest metrics: win rate, avg R:R, max drawdown, Sharpe
- Prediction parameters (instrType, direction, minGrade)
- "Apply to Predictions" button (navigates with params pre-loaded)

STRATEGY BUILDER (Backtest page):

Type plain English: "Buy NIFTY calls when RSI < 35 and price above EMA 20"

AI parses to: instrType, direction, symbol, moduleId, minGrade, filters[]
Shows accuracy impact: baseline vs with-strategy, net delta
Save to My Strategies for reuse

21. Accessibility (WCAG 2.1 AA)

- All interactive elements keyboard-navigable
- All images have alt text
- Colour alone never conveys meaning (bull/bear use text labels too)
- ARIA roles correct (eslint-plugin-jsx-a11y enforced at build time)
- Focus ring visible on all focusable elements
- Mobile touch targets minimum 44px (WCAG requirement)
- Screen reader announcements for live regions (role=alert, aria-live)
- Disclaimer non-removable and non-dismissible
- No FOMO language (ACT NOW, URGENT, Don't miss out)
- Accuracy metrics always show losing signals (no cherry-picking)
- Every signal card shows complement label: X% means ~Y% chance of being wrong

22. Troubleshooting

PROBLEM: Could not reach server

CHECK: Is node start.js --dev running?

CHECK: <http://localhost:5000/api/health> should return JSON

CHECK: Port 5000 not blocked by firewall

PROBLEM: Invalid credentials

CHECK: Username is lowercase (krishna.s not Krishna.s)

CHECK: Password meets requirements (uppercase + number + special)

CHECK: Account not deactivated (check Admin panel)

PROBLEM: Invalid key / Key expired

FIX: Generate new key: `npm run keygen <username>`

OR: Use "Get Key" button on login Step 2 page

PROBLEM: Step token expired

FIX: Go back to Step 1 and log in again (10-minute window)

PROBLEM: Port already in use (EADDRINUSE)

FIX: start.js automatically frees ports on startup

MANUAL: `taskkill /F /PID <pid>` (Windows)

PROBLEM: AI backend not starting (ModuleNotFoundError)

FIX: `ai_backend/venv/Scripts/python.exe -m pip install -r ai_backend/requirements.txt`

NOTE: App works without AI backend (JS engine fallback)

PROBLEM: Data password forgotten

WARNING: Data cannot be recovered without the password

FIX: Delete data/ folder and start fresh (all predictions lost)

PROBLEM: XGBoost not installed warning

FIX: `ai_backend/venv/Scripts/python.exe -m pip install xgboost==2.1.3`

NOTE: Non-critical, AI uses mock predictions as fallback